
Как управлять контекстом, если вы постоянно упираетесь в лимиты

*Практическое руководство: как сократить расход
токенов
на 60–80% без потери качества
на основе 100+ источников*

t.me/prod1337

Практический гайд по Claude Code

*На основе официальной документации Anthropic,
открытых исследований и реальных кейсов*

Май 2026

Содержание

ВВЕДЕНИЕ

- 1 Введение: почему модель тупеет и работает медленно

ОСНОВНОЙ МАТЕРИАЛ

- 2 Часть 1: сначала измеряй, потом режь
- 3 Часть 2: как перестать платить за «вечно включённые» правила
- 4 Часть 3: как не тащить за собой историю всей жизни
- 5 Часть 4: как перестать скормливать модели простыни логов
- 6 Часть 5: как делегировать черновую работу «дешёвому стажёру»
- 7 Часть 6: когда инструменты вредят больше, чем помогают
- 8 Часть 7: как заставить модель «думать» ровно столько, сколько нужно
- 9 Часть 8: как заставить модель запоминать, а не перечитывать
- 10 Часть 9: когда НЕ нужно ничего оптимизировать
- 11 Часть 10: как понять, что что-то пошло не так

ФИНАЛ

- 12 Финальное напутствие
- 13 Присоединяйся к сообществу

Введение: почему модель тупеет и работает медленно

Ты открываешь Claude Code, чтобы исправить небольшой баг в React-компоненте. Первые 15 минут всё работает идеально: модель быстро читает код, предлагает точные решения, пишет тесты.

Но через час что-то ломается. Модель начинает отвечать по 30 секунд. Вместо того чтобы исправить одну строчку, она переписывает весь файл. Она забывает правила проекта, которые ты объяснял в начале сессии. Если ты на платной подписке — ты упёрся в лимит и ждёшь сброса. Если на API — смотришь на счёт и видишь 5 долларов за простую задачу.

Почему это происходит? Потому что модель захлёбывается от объёма информации.

Каждый раз, когда ты просишь Claude Code что-то сделать, он отправляет на сервер не только твой вопрос. Он отправляет всю историю переписки, все системные правила, все схемы подключённых инструментов и результаты всех команд, которые он выполнил. Этот «контекстный налог» растёт с каждым шагом.

Когда контекст раздувается до 150 000 токенов, происходят три вещи:

1. **Модель теряет фокус.** Она видит 150 000 слов и пытается учесть их все. Важная инструкция «используй TypeScript» теряется где-то между логами Docker и историей обсуждения прошлой фичи.
2. **Ты платишь за мусор.** Каждый токен стоит денег или лимита. Если модель читает 5000 строк логов `npm test`, чтобы найти одну ошибку на 145-й строке, ты платишь за 4999 строк, которые ей не нужны.
3. **Скорость падает.** Серверу нужно время, чтобы прочитать и обработать этот огромный массив данных.

Этот гайд не про то, какие команды вводить в терминале. Он про то, как управлять вниманием модели: давать ей ровно ту информацию, которая нужна для текущей задачи, и отсекал всё остальное.

Результат: модель снова становится умной, работает в два раза быстрее, а счета за API и расход лимитов падают на 60–80%.

Часть 1: сначала измеряй, потом режь

Зачем тебе измерять контекст

Невозможно оптимизировать то, что ты не видишь. Прежде чем удалять файлы или менять настройки, нужно понять, куда именно утекают твои токены и деньги. Часто проблема кроется не там, где ты думаешь: возможно, это не длинный код, а какой-то скрытый плагин, который на каждый твой чих отправляет модели мегабайт логов.

Что произойдёт, если действовать вслепую

Если ты начнёшь удалять всё подряд без измерений, ты можешь удалить полезные правила проекта, но оставить включённым тяжёлый инструмент. В итоге модель станет глупее (потому что забыла правила), но работать быстрее не начнёт. Ты потратишь время на рефакторинг `CLAUDE.md`, а проблема окажется в том, что модель читает папку `node_modules`.

Как это исправить

Сделай аудит своей среды перед началом работы. Для этого не нужны сложные дашборды, достаточно пары команд.

1. **Используй** `/context` и `/usage`. Эти встроенные команды покажут тебе, сколько токенов загружено прямо сейчас и на что они потрачены. Если ты только открыл проект, а у тебя уже занято 20 000 токенов — это повод насторожиться.
2. **Проверь вес файлов.** Обычная команда `wc -w CLAUDE.md` покажет, сколько слов в твоих правилах. Если их больше 300 — пора резать.
3. **Ищи скрытый мусор.** Иногда логи инструментов или кэши генерируют гигантские файлы. Используй команду `find . -name "*.log" -size +1M`, чтобы найти и удалить их, пока модель случайно их не прочитала.

Что изменится

Ты будешь точно знать свой «бюджет внимания». Ты увидишь, что отключение одной ненужной интеграции даёт больше экономии, чем удаление абзаца текста из правил. Оптимизация станет хирургической, а не случайной.

Часть 2: как перестать платить за «вечно включённые» правила

Зачем тебе файл CLAUDE.md

Когда начинаешь новый проект, хочется, чтобы модель сразу понимала, как здесь всё устроено. Ты создаёшь файл `CLAUDE.md` и пишешь туда всё: архитектуру, стек технологий, правила именования переменных, как запускать тесты, как деплоить проект, кто отвечает за какие модули.

В итоге получается документ на 500 строк. Кажется, что всё сделано правильно — теперь модель знает всё.

Что произойдёт, если оставить всё как есть

Файл `CLAUDE.md` — это «вечно включённый» контекст. Модель читает его **при каждом** запросе.

Если ты просишь модель переименовать переменную в одном файле, она сначала читает 500 строк про архитектуру, деплой и правила написания тестов. А потом уже переименовывает переменную.

Если ты делаешь 50 запросов за сессию, модель прочитает эти 500 строк 50 раз. Ты заплатишь за это 50 раз. А самое главное — из-за того, что правил слишком много, модель начнёт их игнорировать. Она не сможет отличить критически важное правило «не используй `any` в TypeScript» от справочной информации «деплой происходит через GitHub Actions».

Как это исправить

Раздели информацию на две категории: **правила** и **справочник**.

В `CLAUDE.md` должны остаться только те правила, которые модель должна применять при написании каждой строчки кода. Это твой «конституционный минимум».

Всё остальное — справочная информация, архитектурные решения, гайды по деплою — должно лежать в отдельных файлах (например, в папке `docs/`). Когда модели понадобится узнать, как деплоить проект, она сама найдёт и прочитает файл `docs/deployment.md`. Ей не нужно держать это в голове постоянно.

Что должно быть в CLAUDE.md:

- Строгие запреты (чего делать категорически нельзя)
- Базовый стек технологий (React 18, TypeScript, Tailwind)
- Стилль кода (функциональные компоненты, а не классы)
- Паттерны обработки ошибок

Что нужно убрать из CLAUDE.md:

- Длинные примеры кода (оставь только короткие паттерны)
- Инструкции по настройке окружения (это нужно только один раз при старте)
- Историю принятия архитектурных решений
- Списки всех доступных скриптов (модель сама может посмотреть `package.json`)

Что изменится

Твой `CLAUDE.md` сократится с 500 до 100 строк. Модель перестанет отвлекаться на нерелевантную информацию. Когда ты попросишь её написать компонент, она чётко вспомнит правило про TypeScript, потому что оно не затерялось среди инструкций по деплою. И ты сэкономишь тысячи токенов — или лимита — на каждом запросе.

Наглядный пример: мусор в CLAUDE.md

Вот как выглядит типичный файл с правилами, который разработчики создают в первый день проекта. Кажется, что здесь всё полезно, но на самом деле 80% этого текста — чистый яд для внимания модели.

✗ Как делать не надо — токсичный CLAUDE.md

```
# Правила проекта E-commerce Store

## Контакты и ресурсы
Project Lead: john@company.com
Документация: https://docs.company.com
Slack Channel: #e-commerce-team
Jira Board: https://jira.company.com/e-commerce

## Архитектура
Мы используем микросервисную архитектуру. Frontend написан на React 18,
Backend на Node.js. База данных PostgreSQL. В качестве кэша используем Redis.
Исторически сложилось так, что мы перешли с Angular на React в 2023 году,
поэтому в папке legacy/ ещё остались старые компоненты. Пожалуйста, не трогай их.

## Как собирать проект
npm run build          # Сборка для продакшена
npm run build:dev       # Сборка для разработки
npm run deploy:staging  # Деплой на тестовый сервер
npm run deploy:production # Деплой на бой
npm run migrate         # Запуск миграций базы данных
npm run migrate:rollback # Откат последней миграции

## Оптимизация производительности (Важно!)
- Используй Code splitting с помощью React.lazy
- Оптимизируй картинки через next/image
- Используй мемоизацию (React.memo) для тяжёлых компонентов
- Настраивай ленивую загрузку для маршрутов
- Анализируй размер бандла через webpack-bundle-analyzer

## Базовые правила кода
- Используй TypeScript
- Не используй any
- Называй интерфейсы с буквы I (например, IUserService)
- Пиши тесты на Jest
```

Почему это плохо: модель читает этот текст каждый раз, когда ты просишь её поправить опечатку в кнопке. Ей абсолютно не нужно знать email Джона или команду отката миграций, чтобы изменить цвет текста. Правило «не используй `any`» погребено в самом конце, и модель с вероятностью 50% его проигнорирует, устав читать про историю перехода с Angular.

✓ Как надо — чистый CLAUDE.md

```
# Core Rules
- Stack: React 18, TypeScript, Tailwind.
- Types: NEVER use `any`. Prefix interfaces with `I` (e.g., `IUser`).
- Components: Functional only. Use `React.memo` for lists.
- Testing: Jest. Every new component must have a `.test.tsx` file.

# Context Pointers
- Legacy code is in `legacy/`. Do not modify it unless explicitly asked.
- For architecture decisions, read `docs/architecture.md`.
- For deployment scripts, check `package.json`.
```

Почему это хорошо: только то, что влияет на написание кода прямо сейчас. Никаких контактов, никаких простыней с командами. Если модели понадобится узнать архитектуру, она самаходит и прочитает указанный файл.

Часть 3: как не тащить за собой историю всей жизни

Зачем тебе контекст переписки

Когда работаешь с моделью, тебе важно, чтобы она помнила, о чём вы говорили 10 минут назад. Ты попросил написать функцию, потом добавить обработку ошибок, потом написать тест. Модель помнит весь этот контекст, и это создаёт ощущение непрерывного диалога с умным напарником.

Что произойдёт, если оставить всё как есть

Проблема в том, что задачи меняются. Допустим, первый час вы вместе разбирались со сложным багом в авторизации. Вы перепробовали пять разных подходов, прочитали десятки файлов, посмотрели логи. Баг исправили.

Теперь ты переходишь к следующей задаче — нужно поправить вёрстку кнопки в корзине.

Если просто продолжить диалог, модель потащит с собой весь этот мусор про авторизацию. Когда она будет править кнопку, в её «голове» всё ещё будут лежать токены авторизации, старые логи и неудачные попытки исправить баг.

Это как пытаться решать задачу по геометрии, пока кто-то кричит тебе в ухо правила орфографии. Модель начинает путаться, предлагать странные решения, а скорость её ответов падает до минимума, потому что ей нужно обрабатывать весь этот массив прошлой жизни. Плюс ты сжигаешь лимиты или деньги на токены, которые уже не нужны.

Как это исправить

Введи привычку «закрывать фазы». Как только одна логическая задача завершена (баг исправлен, фича написана, тесты прошли), нужно очистить контекст.

В Claude Code для этого есть команда `/compact`. Она делает гениальную вещь: не просто удаляет историю, а просит саму модель написать краткое резюме того, что было сделано, и текущего состояния проекта.

Вместо 50 сообщений с логами, ошибками и попытками, в контексте остаётся один абзац: «Мы исправили баг в авторизации, обновив JWT-токен в файле `auth.ts`. Сейчас всё работает, тесты зелёные».

Когда нужно сжимать контекст:

- После того как тесты успешно прошли
- Перед тем как переключиться на новую фичу
- Когда модель начинает «тупить» и предлагать странные решения
- Когда ответ модели занимает больше 15–20 секунд
- Когда ты видишь, что контекст заполнен больше чем на 50%

Что изменится

Ты будешь начинать каждую новую задачу с «чистого листа», но при этом модель не забудет глобальный контекст того, над чем вы работаете. Её ответы снова станут мгновенными и точными. Ты перестанешь платить за то, чтобы модель перечитывала ваши старые ошибки.

Часть 4: как перестать скармливать модели простыни логов

Зачем модели нужны логи инструментов

Чтобы модель могла исправлять ошибки, ей нужно видеть, что пошло не так. Она запускает тесты (`npm test`), компилирует код (`npm run build`) или ищет файлы (`grep`). Инструменты выдают результат, модель его читает и понимает, где проблема.

Что произойдёт, если оставить всё как есть

Большинство инструментов создавались для того, чтобы их читал человек, а не нейросеть.

Когда запускаешь `npm test` в большом проекте, инструмент может выдать 5000 строк текста. Там будут красивые прогресс-бары, списки всех успешно пройденных тестов, предупреждения о старых версиях библиотек и где-то в самом конце — те 10 строк с ошибкой, ради которых всё и затевалось.

Человек просто проскроллит этот текст вниз и посмотрит на ошибку. Модель так не умеет. Она честно прочитает все 5000 строк. Потратит на это твои деньги, своё «внимание» и время. А самое страшное — ошибка может просто затеряться в этом объёме текста, и модель сделает неправильный вывод.

То же самое происходит с поиском по коду. Если модель ищет, где используется функция, и получает в ответ 10 000 строк из скомпилированных файлов или папки `node_modules`, она захлёбнётся в этом мусоре.

Как это исправить

Никогда не позволяй модели читать сырой вывод инструментов. Всегда фильтруй информацию до того, как она попадёт в контекст.

Вместо того чтобы просто запускать тесты, научи модель (через тот же `CLAUDE.md`) запускать их так, чтобы они показывали только ошибки.

Если модель ищет файлы, она должна исключать папки с зависимостями и скомпилированным кодом.

Если она собирает проект, ей нужны только последние 20 строк лога, где обычно и находится причина падения сборки.

Как это выглядит на практике:

- Вместо «запусти тесты» → «запусти тесты и покажи только упавшие»
- Вместо «найди все упоминания функции» → «найди упоминания функции, исключая тесты и `node_modules`, и покажи только имена файлов»
- Вместо «покажи лог коммитов» → «покажи последние 5 коммитов»

Что изменится

Объём информации от инструментов сократится на 90–95%. Модель будет получать только концентрат: «Упал один тест на строке 145, ожидалось 200, пришло 500». Это всё, что ей нужно для работы. Скорость решения проблем вырастет кратно, потому что модели больше не нужно искать иголку в стоге сена.

Наглядный пример: простыни логов

Ты попросил модель починить упавший тест. Она запускает команду `npm test` и получает результат. Посмотри, что она видит.

❌ Как делать не надо — сырой вывод инструмента

```
> e-commerce-app@1.0.0 test
> jest

PASS  src/components/Header.test.tsx (45ms)
PASS  src/components/Footer.test.tsx (32ms)
PASS  src/components/ProductCard.test.tsx (28ms)
PASS  src/components/CartItem.test.tsx (35ms)
PASS  src/components/UserMenu.test.tsx (42ms)
PASS  src/utils/formatCurrency.test.ts (15ms)
PASS  src/utils/calculateTax.test.ts (12ms)
PASS  src/hooks/useCart.test.ts (25ms)
PASS  src/hooks/useAuth.test.ts (30ms)
PASS  src/api/products.test.ts (55ms)
PASS  src/api/users.test.ts (48ms)
PASS  src/store/cartSlice.test.ts (22ms)
PASS  src/store/userSlice.test.ts (20ms)

FAIL  src/components/CheckoutForm.test.tsx (2341ms)
  • CheckoutForm › should submit order successfully

    Expected status code 200, but received 500.
    Server responded with: "Missing required field: zipCode"

      45 |     await userEvent.click(submitButton);
      46 |
    > 47 |     expect(mockSubmit).toHaveBeenCalledWith(200);
          |                                     ^
      48 |   });

at Object.<anonymous> (src/components/CheckoutForm.test.tsx:47:24)

PASS  src/components/AddressInput.test.tsx (38ms)
PASS  src/components/PaymentMethod.test.tsx (41ms)
PASS  src/utils/validateEmail.test.ts (10ms)

Test Suites: 16 passed, 1 failed, 17 total
Tests:      84 passed, 1 failed, 85 total
Time:       12.543 s
```

Почему это плохо: в реальном проекте здесь будет не 17 файлов, а 500. Модель прочитает тысячи строк со словом «PASS», прежде чем доберётся до единственной ошибки. Это сжигает токены, время и внимание. Модель может просто «устать» и сделать неправильный вывод о причине ошибки.

✅ Как надо — отфильтрованный вывод

Вместо того чтобы разрешать модели запускать просто `npm test`, ты настраиваешь в правилах или хуках запуск с фильтрацией:

```
FAIL src/components/CheckoutForm.test.tsx
  • CheckoutForm › should submit order successfully

    Expected status code 200, but received 500.
    Server responded with: "Missing required field: zipCode"
    at src/components/CheckoutForm.test.tsx:47:24
```

Test Summary: 1 failed out of 85 tests.

Почему это хорошо: модель получает только суть. Ошибка на 47-й строке, не хватает поля `zipCode`. Решение очевидно с первой секунды. Ты сэконобил 95% токенов на этом запросе.

Часть 5: как делегировать черновую работу «дешёвому стажёру»

Зачем тебе нужны все файлы проекта

Иногда задача требует понимания широкого контекста. Например, нужно написать документацию по всем API-эндпоинтам в проекте. Или найти, почему стили ломаются на разных страницах. Модели нужно прочитать десятки файлов, чтобы собрать картину воедино.

Что произойдёт, если оставить всё как есть

Ты просишь Claude Code проанализировать 50 файлов. Модель загружает их все в свой контекст. Это может занять 100 000 токенов.

Claude Code (особенно если он использует мощную модель вроде Claude Sonnet или Opus) — это дорогой и умный «сеньор». Заставлять его читать 50 файлов только для того, чтобы найти там список эндпоинтов — это как нанять архитектора с зарплатой \$200 в час, чтобы он переписывал данные из одной Excel-таблицы в другую.

Ты заплатишь огромные деньги (или сожжёшь лимиты) за простую задачу извлечения данных. А потом этот огромный объём прочитанных файлов останется в контексте и будет тормозить всю дальнейшую работу.

Как это исправить

Используй подход «дешёвого стажёра» (cheap worker).

У тебя есть доступ к более быстрым и дешёвым моделям (например, Claude Haiku). Эта модель стоит копейки и работает мгновенно. Она не сможет спроектировать сложную архитектуру, но она идеально справляется с задачей «прочитай эти 50 файлов и выпиши из них все API-эндпоинты».

Вместо того чтобы грузить все файлы в основную сессию, ты открываешь отдельную вкладку терминала. Запускаешь там «стажёра» (Haiku) и даёшь ему простую инструкцию: «Прочитай папку `src/api` и сделай краткую выжимку всех маршрутов в файл `api_summary.md`».

Стажёр делает грязную работу, читает 100 000 токенов (что стоит копейки), и создаёт компактный файл с нужными данными.

Затем ты возвращаешься к своему основному «сеньору» (Sonnet) и говоришь: «Прочитай `api_summary.md` и на его основе напиши документацию».

Что изменится

Ты изолируешь «шумную» работу от основной сессии. Твой основной контекст остаётся чистым и сфокусированным. Ты получаешь тот же результат, но платишь за него в 10 раз меньше и не засоряешь «мозги» основной модели ненужными деталями реализации 50 файлов.

Наглядный пример: делегирование «дешёвому стажёру»

Тебе нужно составить список всех маршрутов (роутов) в старом проекте, чтобы написать для них документацию.

✗ Как делать не надо — грузим «сеньора»

Ты открываешь основную сессию Claude Code (где работает дорогая и умная модель Claude Sonnet) и пишешь:

«Прочитай все файлы в папке `src/pages` и `src/api` и выпиши мне все доступные URL».

Модель начинает послушно открывать файл за файлом:

- Открывает `src/pages/Home.tsx` (читает 500 строк вёрстки, чтобы найти один URL `/home`)
- Открывает `src/pages/Profile.tsx` (читает 800 строк логики профиля, чтобы найти `/profile`)
- Открывает `src/api/orders.ts` (читает 1000 строк SQL-запросов, чтобы найти `/api/orders`)

Она тратит 150 000 токенов, читая бизнес-логику, стили и SQL-запросы, которые ей вообще не нужны для этой задачи. Ты платишь за это несколько долларов. А главное — теперь все эти стили и SQL-запросы навсегда остались в контексте вашей беседы. Когда ты попросишь её написать новый компонент, она может случайно использовать кусок старого SQL-запроса, потому что он «застрял» у неё в памяти.

Важное отличие: стажёр (cheap coworker) vs изолятор (subagent)

Не путай делегирование дешёвой модели с использованием **субагентов**.

«Дешёвый стажёр» (cheap coworker) — это когда ты просишь быструю модель (например, Haiku) прочитать много текста и сделать выжимку. Экономия идёт за счёт разницы в цене между моделями.

Субагент — это когда ты поручаешь отдельному, изолированному процессу выполнить грязную работу (например, «найди причину падения тестов в этом модуле»). Субагент работает в своём собственном контексте. Он может прочитать 50 файлов, наделать ошибок, перечитать логи, но **в твоей основной контекст** вернётся только финальный ответ: «Проблема в строке 42, вот патч».

Субагенты изолируют мусор. Они не всегда экономят деньги (иногда они тратят больше токенов на свои размышления), но они спасают внимание твоего основного агента. Твой главный контекст остаётся кристально чистым.

✓ Как надо — используем «стажёра»

Ты открываешь соседнее окно терминала. Запускаешь там дешёвую и быструю модель (например, Claude Haiku).

«Быстро просканируй папку `src/`, найди все строки, где упоминаются URL-адреса или роуты, и сохрани их списком в файл `routes_summary.md`. Саму логику файлов игнорируй».

Haiku пролетает по файлам за пару секунд. Это стоит тебе 5 центов. В корне проекта появляется аккуратный файл `routes_summary.md`:

```
# Найденные маршруты
- `/home` (src/pages/Home.tsx)
- `/profile` (src/pages/Profile.tsx)
- `/api/orders` (src/api/orders.ts)
```

Затем ты возвращаешься в своё основное окно к умному «сеньору» (Sonnet) и говоришь:

«Прочитай файл `routes_summary.md` и на его основе напиши красивую документацию в формате OpenAPI».

Почему это хорошо: твоя основная сессия осталась кристально чистой. Умная модель не видела грязного старого кода, она увидела только чистый список роутов и идеально выполнила свою архитектурную задачу.

Часть 6: когда инструменты вредят больше, чем помогают

Зачем нужны интеграции (MCP и хуки)

Современные AI-ассистенты умеют подключаться к внешним системам. Они могут читать твой GitHub, проверять Jira, смотреть статистику в Sentry или дёргать API внутренних сервисов компании. Это называется MCP (Model Context Protocol).

Кажется логичным подключить всё, что есть. Чем больше у модели инструментов, тем она умнее, верно?

Что произойдёт, если оставить всё как есть

Каждый подключённый инструмент — это инструкция, которую модель должна держать в голове. «У тебя есть инструмент Jira. Чтобы им воспользоваться, тебе нужно передать ID задачи. Он вернёт тебе статус и описание».

Если ты подключил 10 инструментов, модель постоянно тратит часть своего внимания на то, чтобы помнить, как ими пользоваться. Даже если сейчас ты просто просишь её поправить CSS-стиль кнопки.

Хуже того, иногда инструменты работают «в фоне» (это называется хуки). Например, при каждом твоём запросе инструмент может автоматически прикреплять информацию о текущей ветке git или последние изменения в файлах. Это добавляет скрытый мусор в каждый твой запрос. Модель начинает путаться: «Почему мне прислали git diff, если я просто попросил переименовать переменную?»

Как это исправить

Относись к инструментам модели как к приложениям на телефоне: удаляй всё, чем не пользуешься каждый день.

Если ты не работаешь с Jira в этой конкретной сессии — отключи этот инструмент. Если тебе не нужно, чтобы модель сама делала коммиты — отключи интеграцию с git.

Оставь только базовый набор: чтение файлов, выполнение команд в терминале, редактирование кода. Подключай сложные инструменты (аналитика, базы данных, таск-трекеры) только тогда, когда задача напрямую этого требует.

Что изменится

Модель перестанет галлюцинировать и пытаться использовать инструменты там, где они не нужны. Её инструкции станут короче, а значит, она будет лучше фокусироваться на самом коде. Ты перестанешь платить за «схемы использования» инструментов, которые тебе сейчас не нужны.

Часть 7: как заставить модель «думать» ровно столько, сколько нужно

Зачем нужны уровни «размышления»

Иногда задача кажется простой, но требует глубокого анализа. Например, «почему этот компонент рендерится дважды?». Чтобы ответить на этот вопрос, модели нужно проследить цепочку вызовов, понять жизненный цикл React и найти неочевидную зависимость.

Для таких случаев модели научились «думать» перед тем, как ответить (reasoning effort). Они генерируют скрытые токены размышлений, анализируют разные варианты и только потом выдают результат.

Что произойдёт, если оставить всё как есть

Если включить максимальный уровень размышлений для всех задач, модель начнёт философствовать там, где нужно просто действовать.

Ты просишь её: «Переименуй функцию `getUser` в `fetchUser`».

Модель с высоким уровнем размышления начинает: «Так, пользователь просит переименовать функцию. Зачем? Возможно, это связано с паттернами работы с сетью. Если я переименую её здесь, не сломает ли это что-то в другом месте? Давайте проанализируем все файлы...».

Вместо 2 секунд и 1 цента, ты ждёшь 20 секунд и платишь 10 центов за простейшую операцию поиска и замены.

С другой стороны, если полностью отключить размышления, модель начнёт ошибаться в сложных задачах, пытаясь выдать ответ с ходу, без анализа последствий.

Как это исправить

Управляй уровнем размышлений в зависимости от задачи. Это как с людьми: ты не просишь архитектора писать стратегию развития на 5 лет, когда нужно просто вкрутить лампочку.

Для 80% повседневных задач (написать тест, добавить кнопку, поправить опечатку) модели вообще не нужно глубоко «думать». Она и так знает паттерны.

Включай «глубокое размышление» только тогда, когда:

- Ты просишь спроектировать архитектуру новой фичи
- Ты не можешь найти причину плавающего бага
- Нужно провести ревью безопасности кода
- Нужно сделать сложный рефакторинг с множеством зависимостей

Что изменится

Ты найдёшь баланс между скоростью, ценой и качеством. Простые задачи будут решаться мгновенно и дёшево. Сложные задачи будут решаться вдумчиво и качественно, без обидных ошибок из-за спешки.

Часть 8: как заставить модель запоминать, а не перечитывать

Зачем нужно кэширование

Каждый раз, когда ты отправляешь запрос, модель заново читает твои системные правила, схемы инструментов и базовый контекст. И каждый раз ты за это платишь.

Если твой базовый контекст (правила + инструменты + структура проекта) весит 10 000 токенов, то за 50 запросов в течение дня модель прочитает 500 000 токенов только базовой информации. Это не просто дорого — это медленно. Чтение 10 000 токенов каждый раз занимает время сервера. Твои запросы будут обрабатываться с задержкой, даже если ты просишь сделать простейшую вещь.

Как это исправить

Современные модели поддерживают технологию Prompt Caching (кэширование промптов). Суть проста: если часть твоего запроса не меняется, сервер сохраняет её в быструю память. В следующий раз он не читает её заново, а просто достаёт из памяти. Это стоит на 90% дешевле и работает в разы быстрее.

Но чтобы эта магия сработала, нужно правильно структурировать информацию.

Сервер кэширует информацию только в том случае, если она находится **в самом начале** запроса и **абсолютно не меняется**. Как только сервер встречает изменённый текст, он перестаёт кэшировать всё, что идёт после него.

Как сломать кэширование (как делать не надо):

Ты помещаешь текущую задачу («исправь баг в корзине») в начало, а системные правила — в конец. Так как текущая задача меняется каждый раз, сервер видит изменения в самом начале и перечитывает весь текст заново. Кэш не работает.

Как починить кэширование (как надо делать):

Всегда помещай самую стабильную информацию в начало:

1. Сначала идут системные правила (они не меняются).
2. Затем схемы инструментов (они не меняются).
3. Затем архитектура проекта (она почти не меняется).
4. И только в самом конце — текущая задача, логи ошибок и история диалога (они меняются постоянно).

Что изменится

Как только ты структурируешь контекст правильно, магия заработает. Твои счета за API упадут, потому что ты будешь платить 10% от стоимости за чтение стабильной части. А ответы модели станут почти мгновенными, так как серверу нужно будет прочитать только пару свежих абзацев в конце твоего запроса.

Наглядный пример: правильное кэширование

Сервер пытается понять, можно ли взять часть запроса из кэша, чтобы не читать заново.

Как делать не надо — кэш ломается

```
[ДИНАМИЧЕСКАЯ ЧАСТЬ — МЕНЯЕТСЯ КАЖДЫЙ РАЗ]
Текущее время: 14:35:22
Текущая задача: Добавь отступы в шапку сайта.
Последняя ошибка: TypeError: Cannot read properties of undefined
```

```
[СТАБИЛЬНАЯ ЧАСТЬ — НЕ МЕНЯЕТСЯ]
Системные инструкции: Ты эксперт по React...
Правила проекта: Используй Tailwind, не используй inline-стили...
Доступные инструменты: [Схема инструмента File], [Схема инструмента Bash]...
```

Почему это плохо: сервер читает запрос сверху вниз. В первой же строчке он видит новое время и новую задачу. Он понимает: «Ага, текст изменился!». И с этого момента он честно читает всё до самого конца, включая огромные схемы инструментов и правила. Кэширование не сработало.

Как надо — кэш работает

```
[СТАБИЛЬНАЯ ЧАСТЬ — НЕ МЕНЯЕТСЯ]
Системные инструкции: Ты эксперт по React...
Правила проекта: Используй Tailwind, не используй inline-стили...
Доступные инструменты: [Схема инструмента File], [Схема инструмента Bash]...
```

```
[ДИНАМИЧЕСКАЯ ЧАСТЬ — МЕНЯЕТСЯ КАЖДЫЙ РАЗ]
Текущее время: 14:35:22
Текущая задача: Добавь отступы в шапку сайта.
Последняя ошибка: TypeError: Cannot read properties of undefined
```

Почему это хорошо: сервер начинает читать сверху. Он видит системные инструкции и схемы инструментов. «О, я это уже видел 5 минут назад!» — думает сервер. Он мгновенно достаёт эти 10 000 токенов из кэша (что стоит копейки). И только дойдя до конца, он видит новую задачу. Ему остаётся прочитать всего 50 новых токенов. Результат — мгновенный и дешёвый ответ.

Часть 9: когда НЕ нужно ничего оптимизировать

Зачем вообще отказываться от оптимизации

Весь этот гайд рассказывает о том, как урезать, фильтровать и сжимать информацию. Кажется, что это нужно делать всегда. Но в погоне за экономией токенов можно потерять главное — качество решений.

Что произойдёт, если оптимизировать всё подряд

Допустим, у тебя упал продакшен-сервер. Клиенты не могут оплатить заказы. Каждая минута простоя стоит компании тысячи долларов.

Ты открываешь Claude Code и пытаешься найти причину. Но из-за строгих правил оптимизации модель не видит полные логи сервера, а только отфильтрованные ошибки. Она не видит историю изменений в базе данных, потому что ты отключил этот инструмент. Она не может найти неочевидную связь между недавним коммитом и падением базы, потому что ты сжал контекст.

В итоге ты сэкономил 50 центов на токенах, но потерял часы на поиск проблемы, потому что модель была «слепой».

Как понять, когда остановиться

Оптимизация контекста — это инструмент для повседневной рутины. Для тех 80% задач, которые понятны, предсказуемы и не несут огромных рисков.

Но есть ситуации, когда нужно «выкрутить ручки на максимум» и дать модели абсолютно всё, что у тебя есть.

Сценарии, когда нужно отключить фильтры и дать полный контекст:

- 1. Инциденты на продакшене.** Когда всё горит, тебе нужен умный напарник с полной картиной происходящего. Загружай полные логи, подключай все инструменты аналитики, давай доступ ко всей истории коммитов. Цена токенов здесь не имеет значения.
- 2. Ревью безопасности.** Если ты просишь модель найти уязвимости в коде авторизации, ей нужно видеть весь поток данных. Отфильтрованный лог может скрыть именно ту деталь, через которую хакеры смогут взломать систему.

3. Проектирование новой архитектуры. Когда ты закладываешь фундамент проекта на годы вперёд, модели нужно понимать бизнес-логику, планы масштабирования и ограничения старой системы. Здесь «вода» может оказаться важным контекстом.

Что изменится

Ты перестанешь воспринимать оптимизацию как религию. Это просто инструмент управления бюджетом и фокусом. Ты научишься переключать режимы: от «бережливого стажёра» для рутины до «дорогого архитектора с полным доступом» для критических задач.

Часть 10: как понять, что что-то пошло не так

Даже если ты всё сделал правильно, иногда система начинает вести себя странно. Вот как распознать симптомы и вылечить их.

Симптом 1: модель вдруг стала писать плохой код

Как это выглядит: Вчера модель писала идеальные React-компоненты, а сегодня начала предлагать устаревшие подходы, забывать про типизацию и игнорировать замечания.

В чём реальная проблема: Ты переборщил с компакцией. Модель забыла важные нюансы, к которым вы пришли в ходе долгих обсуждений, потому что при сжатии они показались «незначительными деталями».

Как лечить: Вернись на шаг назад. Добавь в `CLAUDE.md` те выводы, которые модель постоянно забывает. Если это специфика конкретной задачи — просто опиши её заново в текущем запросе.

Симптом 2: ответы стали занимать по минуте

Как это выглядит: Ты просишь добавить консоль-лог, а модель «думает» 60 секунд.

В чём реальная проблема: У тебя сломалось кэширование, или ты случайно скормил модели огромный файл.

Как лечить: Проверь, не попал ли в начало запроса какой-то динамический текст (например, таймстемп в системном промпте). Проверь, не вывел ли какой-то инструмент гигантский лог, который модель сейчас пытается переварить.

Симптом 3: модель ходит по кругу

Как это выглядит: Модель предлагает решение. Оно не работает. Она извиняется, предлагает другое. Оно тоже не работает. Она извиняется и предлагает первый вариант.

В чём реальная проблема: Контекст переполнен неудачными попытками. Модель видит столько ошибок, что уже не может выстроить чистую логическую цепочку.

Как лечить: Немедленно остановись. Сделай жёсткую компакцию или вообще начни новую сессию. Опиши проблему с чистого листа, приложив только текущее состояние кода и самую последнюю ошибку.

Симптом 4: модель пытается использовать несуществующие инструменты

Как это выглядит: Модель пишет «сейчас я проверю базу данных», хотя у неё нет доступа к базе.

В чём реальная проблема: Ты оставил в системном промпте или в `CLAUDE.md` инструкции по работе с инструментами, которые отключил.

Как лечить: Синхронизируй `CLAUDE.md` с реальным набором подключённых инструментов. Если инструмент отключён — удали все упоминания о нём из правил.

Симптом 5: модель пошла писать код, а ты понял, что забыл уточнить деталь

Как выглядит:

Ты нажал Enter, модель начала генерировать ответ, и тут ты понимаешь: «Блин, я же не сказал ей использовать новый API!».

Почему это опасно:

Если ты дождёшься конца генерации, а потом скажешь «переделай под новый API», в историю запишется и неправильный код, и твоё исправление. Контекст раздуется в два раза на пустом месте.

Лечение:

Прерывай модель немедленно. В Claude Code используй `Esc` (или `Esc+Esc`), чтобы остановить генерацию. Ещё лучше — используй команду `/rewind`, чтобы отмотать историю на шаг назад, словно этого ошибочного запроса вообще не было. Это стирает мусор из истории до того, как он успеет там закрепиться. Для быстрых вопросов, которые вообще не должны попасть в историю (например, «как работает эта функция?»), используй команду `/btw`.

Финальное напутствие

Работать с AI-ассистентом — это не то же самое, что писать код или гуглить решения. Это похоже на управление очень умным, но легко отвлекающимся сотрудником.

Твоя главная задача как разработчика больше не в том, чтобы помнить синтаксис методов массива в JavaScript. Твоя задача — **управлять контекстом**.

Тот, кто научится чётко формулировать задачу, отсекай шум, фильтровать логи и вовремя очищать историю, получит в своё распоряжение неумолимого напарника, который пишет код со скоростью мысли.

А тот, кто будет просто скидывать в чат простыни нечитаемых логов и надеяться, что «AI сам разберётся», будет тратить бюджеты компании и получать в ответ медленный, запутанный и неработающий код.

Выбор за тобой. Начни с очистки своего `CLAUDE.md` прямо сейчас.

Присоединяйся к сообществу

Если этот гайд помог тебе сэкономить деньги на API или ускорить работу с Claude Code, поделись своим опытом.

Telegram: t.me/prod1337

Там мы обсуждаем оптимизацию AI-инструментов, делимся кейсами и помогаем друг другу находить новые способы работать умнее, а не дороже.

Увидимся!